

Collatz Descent Simulator

with δ Analysis

Program Description & User's Guide — v2

`collatz_descent.py`

1. Program Overview

`collatz_descent.py` is a Python command-line tool that analyses the Collatz conjecture through a modular-arithmetic and binary-tree framework. For a given depth m it exhaustively simulates all 2^m residue classes $N \equiv b \pmod{2^m}$ and, for each one, computes the complete set of invariants that determine whether that class ascends, descends, or loops under the compressed Collatz map.

Version 2 adds three major capabilities:

- **NumPy vectorisation:** all 2^m classes are simulated in parallel, giving a 5–6× speedup over the original loop-based engine.
- **Pascal-cell aggregation:** for large m the 2^m raw rows collapse to $m+1$ lines (one per Pascal cell), keeping output readable at any depth.
- **Output control:** `--output` writes to `.csv/.tsv`; `--filter` restricts which rows are shown or saved; `--pascal` and `--full` override the automatic mode switch.

The central quantity is $\delta(N) = f/N - \text{gap}$, where $\text{gap} = 2^m - 3^n$. Its sign tells you the fate of every N in that residue class:

- $\delta < 0$ — the class descends ($M < N$)
- $\delta = 0$ — the class is a loop ($M = N$), occurring only where $N = f \div \text{gap}$
- $\delta > 0$ — the class ascends ($M > N$)

2. Mathematical Background

2.1 The Master Invariant

At every node of the binary Collatz tree the following equation holds exactly:

$$2^m \times M = 3^n \times N + f$$

N is the seed, **M** is the value after *m* steps, **n** is the number of odd steps, and **f** is the accumulated residue built up by those odd steps.

2.2 The Parity Vector

Each residue class traces a unique sequence of Even (E) and Odd (O) steps — its parity vector. The parity vector fully determines *m*, *n*, and *f*:

$$f = \sum_i 3^{(n-1-i)} \cdot 2^{(p_i-1)} \quad (\text{positional formula})$$

$p_0 < p_1 < \dots < p_{\square-1}$ are the 1-indexed positions of O-steps.

2.3 Gap and Group Classification

gap = $2^m - 3^n$ classifies every Pascal cell (*m*, *n*):

- $\text{gap} > 0 \rightarrow$ Group A (descent geometrically possible)
- $\text{gap} < 0 \rightarrow$ Group C (ascent guaranteed regardless of *f* or *N*)
- $\text{gap} = 0 \rightarrow$ impossible (Fundamental Theorem of Arithmetic)

The critical threshold is $n/m < \log_2 / \log_3 \approx 0.631$. Because this ratio is irrational, Group B is always empty.

2.4 The δ Quantity

Dividing the master invariant by *N* gives $M/N = (3^n + f/N) / 2^m$. The descent condition $M < N$ is equivalent to $f/N < \text{gap}$. The signed distance from this boundary is:

$$\delta(N) = f/N - \text{gap}$$

For a given path, the minimum valid *N* is *b* (the residue class index), giving the maximum possible δ . As $N \rightarrow \infty$, $\delta \rightarrow -\text{gap}$. For all Group A cells, $-\text{gap} < 0$, confirming that sufficiently large seeds always descend.

3. Requirements & Installation

- **Python 3.6+** (uses f-strings and `fractions.Fraction`)
- **NumPy** (strongly recommended; install via `pip install numpy`). The program falls back gracefully to pure Python if NumPy is absent.
- No other third-party packages required.
- Tested on Linux, macOS, and Windows.

```
# Install NumPy if not already present
pip install numpy

# Make script executable (Linux / macOS)
chmod +x collatz_descent.py

# Run
python collatz_descent.py --m 6
```

4. Output Modes

The program has two display modes and an automatic switching rule:

Mode	When active	Rows produced
Full table	$m < 10$ (default), or <code>--full</code> flag	One row per residue class (2^m rows)
Pascal aggregation	$m \geq 10$ (default), or <code>--pascal</code> flag	One row per Pascal cell ($m+1$ rows)

Pascal aggregation is the recommended mode for $m \geq 10$. It collapses all 2^m residue classes into $m+1$ lines, one per (m, n) cell of Pascal's triangle, with counts of how many classes in that cell have $\delta < 0$, $\delta = 0$, or $\delta > 0$.

Tip

Use `--pascal --output results.csv` for large m . The full table at $m=20$ would be over one million rows; the Pascal table is always exactly $m+1$ rows regardless of depth.

5. Command-Line Parameters

5.1 Required Parameter

Parameter	Type	Description
<code>--m <int></code>	Required	Row depth. Analyses all 2^m residue classes $N \equiv b \pmod{2^m}$. Higher m gives finer resolution; run time and output scale as $O(2^m)$.

5.2 Output Mode Flags

Parameter	Type	Description
<code>--pascal</code>	Flag	Force Pascal-cell aggregation output ($m+1$ rows). Overrides the automatic mode switch.
<code>--full</code>	Flag	Force full per-residue-class table (2^m rows). Overrides the automatic mode switch. Path strings are included for $m \leq 20$.

Note

`--pascal` and `--full` are mutually exclusive. Using both together raises an error.

5.3 Output Destination

Parameter	Type	Description
<code>--output FILE</code>	Optional	Write the selected table to FILE instead of (or in addition to) stdout. Extension determines format: .csv uses commas, .tsv uses tabs. The summary block always prints to stdout regardless.

5.4 Filtering

Parameter	Type	Description
<code>--filter FILTER</code>	Optional	Restrict which rows are shown or written. Applies to both full-table and Pascal modes. Default: all.

Valid FILTER values:

Value	Rows shown
all	All residue classes (default).
descent	Only classes where $\delta < 0$ at minimum N . In Pascal mode: cells containing at least one descent.
ascent	Only classes where $\delta > 0$ at minimum N . Includes all Group C cells.

Value	Rows shown
loops	Only classes where $\delta = 0$ (loop candidates). In Pascal mode: cells containing at least one loop.
groupA	Only Group A cells (gap > 0).
groupC	Only Group C cells (gap < 0).
newloops	Only loop candidates where $N > 2$ (potential new Collatz cycles). Empty in all tested cases so far.

5.5 Utility Flags

Parameter	Type	Description
<code>--no-summary</code>	Flag	Suppress the summary block that prints after the table. Useful when piping output to another tool.
<code>--help</code>	Flag	Display the built-in help message and exit.

6. Quick-Reference: Common Invocations

```
# Basic run at depth 6 (full table, stdout)
python collatz_descent.py --m 6

# Force Pascal aggregation at any depth
python collatz_descent.py --m 6 --pascal

# Full table saved to CSV (paths included, m <= 20)
python collatz_descent.py --m 6 --full --output results.csv

# Large depth – Pascal aggregation is the default
python collatz_descent.py --m 20

# Show only loop candidates (both known attractors at N=1, N=2)
python collatz_descent.py --m 8 --filter loops

# Show only ascending classes, save to TSV
python collatz_descent.py --m 8 --filter ascent --output ascent.tsv

# Group C cells only, Pascal mode, depth 14
python collatz_descent.py --m 14 --filter groupC --pascal
```

```
# Check for any new loops (N > 2) – expected: no rows
python collatz_descent.py --m 12 --filter newloops

# Large run, summary only (suppress the table)
python collatz_descent.py --m 20 --pascal --no-summary
```

7. Understanding the Output

7.1 Full Table

m = 6 full table (64 rows shown)

```
=====
```

	b	Path	n	f	gap	Grp	$\delta(b)$	Loop
	0	EEEEEE	0	0	63	A	-63	
N=1(attr)	1	OEOEOE	3	37	37	A	0	
	2	EOEOEO	3	74	37	A	0	
N=2(attr)	13	OEOEEE	2	11	55	A	-49.5000	
	7	OOOEOE	4	73	-17	C	24.3000	

Column	Description
b	Original residue class index. $N \equiv b \pmod{2^m}$. Also the smallest valid seed for this path when b is odd.
Path	Parity vector of length m. O = odd step, E = even step. Shown for $m \leq 20$.
n	Number of O-steps. Identifies the Pascal cell (m, n).
f	Accumulated residue (exact integer) from the positional formula.
gap	$2^m - 3^n$. Positive $\rightarrow$ Group A; negative $\rightarrow$ Group C.
Grp	A or C.
$\delta(b)$	δ evaluated at $N = b$ (the minimum valid seed). Integer when exact; 4 decimal places otherwise.
Loop	If $\delta = 0$: the loop seed N and a tag. $N \leq 2$ is labelled (attr); $N > 2$ would be flagged *** NEW LOOP ***.

7.2 Pascal-Cell Aggregation Table

m = 10 Pascal-cell aggregation (1024 residue classes $\rightarrow$ 11 cells shown)

```
=====
```

n	$C(m, n)$	gap	Grp	$\delta < 0$	$\delta = 0$	$\delta > 0$	loops
---	-----------	-----	-----	--------------	--------------	--------------	-------

5	252	781	A	250	2	0
N=1 (attr), N=2 (attr)						
6	210	295	A	210	0	0
7	120	-1163	C	0	0	120

Column	Description
n	Odd-step count. Together with the row's m, identifies the Pascal cell.
C(m,n)	Binomial coefficient — total number of residue classes in this cell.
gap	$2^m - 3^n$ for this cell.
Grp	A (gap > 0) or C (gap < 0).
$\delta < 0$	Classes in this cell that descend at their minimum N.
$\delta = 0$	Classes that loop at their minimum N.
$\delta > 0$	Classes that ascend at their minimum N.
loops	Loop seeds found in this cell, if any.

7.3 CSV / TSV Column Reference

When --output is used, the file header varies by mode:

Full table: b, path, n, f, gap, group, delta_num, delta_den, delta_float, loop_N

Pascal mode: m, n, C(m,n), gap, group, delta_neg, delta_zero, delta_pos, loop_N_values

delta_num and delta_den are the exact numerator and denominator of δ as a rational number.
loop_N_values is a semicolon-separated list of loop seeds for that cell (empty string if none).

7.4 Summary Block

```
Summary    m=10    (1,024 residue classes total)
Engine      : NumPy
 $\delta < 0$  descent :      846    ( 82.6%)
 $\delta = 0$  loop   :          2
                n=5  loop_N=1  (attractor)
                n=5  loop_N=2  (attractor)
 $\delta > 0$  ascent  :      176    ( 17.2%)

No new loops found (only known trivial cycle at N≤2).
```

The summary always reflects the full 2^m residue classes regardless of any active `--filter`. The engine line confirms whether NumPy was used.

8. The NumPy Engine

When NumPy is available the simulation runs all 2^m residue classes simultaneously. At each of the m steps, a single boolean mask identifies which classes take an odd step and which take an even step; the f , b , and n arrays are then updated with vectorised operations.

m	Classes	NumPy time	Pure-Python time	Speedup
10	1,024	< 0.001 s	0.002 s	~4×
14	16,384	0.007 s	0.042 s	~6×
18	262,144	0.17 s	0.95 s	~6×
20	1,048,576	~0.7 s	~4 s	~6×

int64 arrays are used for $m \leq 40$, where all values fit safely in 63 bits. For $m > 40$ the engine automatically switches to Python object arrays (arbitrary precision) at the cost of losing vectorisation speed.

δ sign detection uses integer arithmetic throughout ($f - \text{gap} \times b$), avoiding any floating-point division. The Fraction class is only materialised for display in the full table's $\delta(b)$ column.

9. Worked Example

Running at $m = 6$ and then examining the loop candidates:

```
python collatz_descent.py --m 6 --filter loops
```

```
m = 6    full table    (2 rows shown)
  b | Path   |  n |      f |      gap | Grp |       $\delta(b)$  | Loop
  1 | OEEOEOE |   3 |    37 |    37 |   A |           0 |
N=1(attr)
  2 | EOEOEOE |   3 |    74 |    37 |   A |           0 |
N=2(attr)
```

Step-by-step for $b = 1$ (path OEEOEOE):

- $m = 6$, O-positions = {1, 3, 5}, so $n = 3$

- $f = 3^2 \cdot 2^0 + 3^1 \cdot 2^2 + 3^0 \cdot 2^4 = 9 + 12 + 16 = 37$
- $\text{gap} = 2^6 - 3^3 = 64 - 27 = 37$
- $\bar{\delta}(1) = 37/1 - 37 = 0 \rightarrow \text{loop at } N = 1$
- Loop test: $f/\text{gap} = 37/37 = 1 = b \checkmark$

For $b = 13$ (path OEEOOE, a descent case):

- $n = 2$, O-positions = $\{1, 4\}$
- $f = 3^1 \cdot 2^0 + 3^0 \cdot 2^3 = 3 + 8 = 11$
- $\text{gap} = 64 - 9 = 55$
- $\bar{\delta}(13) = 11/13 - 55 = -64.154\dots$ (exact: $-704/13$)
- Integer check: $11 - 55 \times 13 = 11 - 715 = -704 < 0 \rightarrow \text{descent confirmed}$

10. Interpreting the Results

10.1 The $\bar{\delta} = 0$ Cases

Across all tested depths ($m = 1$ to 20), $\bar{\delta} = 0$ occurs only on two alternating paths:

- OEOEOE... (O-first alternating) at $b = 1$, loop at $N = 1$
- EOEOEO... (E-first alternating) at $b = 2$, loop at $N = 2$

Both belong to the known trivial cycle $\{1, 2, 4\}$. No path has ever produced $\bar{\delta} = 0$ with $N > 2$ in any tested row.

10.2 The $\bar{\delta} < 0$ Fraction

The proportion of residue classes with $\bar{\delta}(b) < 0$ grows with m : roughly 62% at $m = 6$, over 80% by $m = 10$ – 14 . The Group C tail (always $\bar{\delta} > 0$) shrinks exponentially because the Pascal weight on $n/m > 0.631$ cells vanishes as m increases.

10.3 Large- N Behaviour

$\bar{\delta}(N) = f/N - \text{gap}$ is monotone decreasing in N . For any Group A path, $\bar{\delta} \rightarrow -\text{gap} < 0$ as $N \rightarrow \infty$. This means even a path with $\bar{\delta}(b) > 0$ at its minimum N will eventually descend for all sufficiently large N in that class. The crossover seed is $N^* = f/\text{gap}$.

11. Performance Guide

m range	Classes	Recommended mode	Typical time (NumPy)
1 – 9	2 – 512	Full table (default)	< 0.01 s
10 – 16	1K – 65K	Pascal aggregation (default)	< 0.1 s
17 – 22	128K – 4M	Pascal aggregation	0.5 – 20 s
23 – 28	8M – 268M	Pascal + --no-summary	minutes; ask for confirmation
> 28	> 268M	--full triggers a confirmation prompt	may be impractical

Warning

Using --full at $m > 28$ will prompt for confirmation before producing hundreds of millions of output rows. Using $m > 30$ also prompts, regardless of mode.

12. Known Limitations

- The program analyses one fixed depth m at a time. A seed's full Collatz orbit spans many depths; this tool characterises the single-step modular behaviour.
- For $m > 40$, f values can exceed int64 range. The NumPy engine falls back to Python object arrays automatically, but loses vectorisation speed.
- Path strings are omitted from the full table when $m > 20$ to keep memory usage manageable.
- Exact δ display uses fractions.Fraction (always correct). For $m > 20$, the sheer number of Fraction computations can slow down the full table mode noticeably; Pascal mode avoids this entirely.

13. Glossary

Term	Definition
m	Total steps (tree depth). Defines the modulus 2^m .
n	Number of odd steps in the parity vector.
b	Residue class index. The smallest valid seed for a given parity vector.
f	Accumulated residue from odd steps, computed by the positional formula.
gap	$2^m - 3^n$. Positive in Group A, negative in Group C.

Term	Definition
Group A	Cells where $\text{gap} > 0$. Descent is possible for sufficiently large N .
Group C	Cells where $\text{gap} < 0$. Ascent is guaranteed regardless of N .
$\delta(N)$	$f/N - \text{gap}$. Negative = descent, zero = loop, positive = ascent.
Pascal cell	The entry (m, n) of Pascal's triangle, containing $C(m,n)$ residue classes.
Parity vector	The E/O sequence for each step, e.g. OEE OEE. Fully determines m , n , and f .
Attractor	The fixed point at $N = 1$ (and $N = 2$ via its even pre-image). The only known loops.
NumPy engine	Vectorised simulation path used when NumPy is installed. $\sim 5\text{--}6\times$ faster.